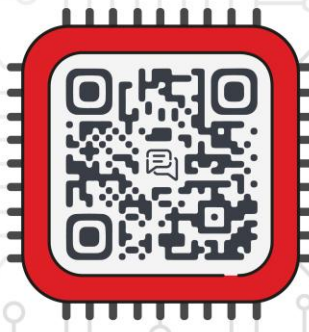




6



17



2550 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

Heap

عملي مشترك



RB Informatics; 20/6/2026

خوارزميات ٢

بسم الله الرحمن الرحيم

في محاضراتنا السابقة تناولنا مقدمة عامة حول بنية الأشجار، ثم انتقلنا إلى دراسة الأشجار الثنائية وخصائصها الأساسية.

بعد ذلك ناقشنا أحد أهم أشكال الأشجار الثنائية وهو شجرة البحث الثنائية (Binary Search Tree) وتعرفنا على كيفية استخدامها في تنظيم البيانات والبحث بكفاءة.

لاحقاً تطرقنا إلى مفهوم التوازن في الأشجار الثنائية، ورأينا كيف يمكن أن يؤثر عدم التوازن على أداء العمليات المختلفة، ودرسنا نموذجاً مهماً من الأشجار المتوازنة وهو شجرة AVL وآلية الحفاظ على التوازن فيها.

واليوم ننتقل إلى نقطة جديدة ضمن هياكل البيانات، وهي بنية تختلف في الهدف وطريقة العمل عن الـ AVL و BST، لكنها تعتمد أيضاً على الشكل الشجري، وهي الـ Binary Heap ، وهي بنية فعالة تُستخدم بكثرة في الخوارزميات وتطبيقات الـ Priority Queue .

ما هو الـ Binary heap ؟

هو بنية بيانات تعتمد على شجرة ثنائية كاملة (Complete Binary Tree) ، حيث تكون جميع المستويات ممتلئة بالكامل ما عدا المستوى الأخير الذي يُملأ من اليسار إلى اليمين. وتمتاز هذه البنية بامتلاكها خاصية تُسمى خاصية الـ Heap

(Heap property) والتي تفرض ترتيباً معيناً بين كل عقدة وأبنائها.

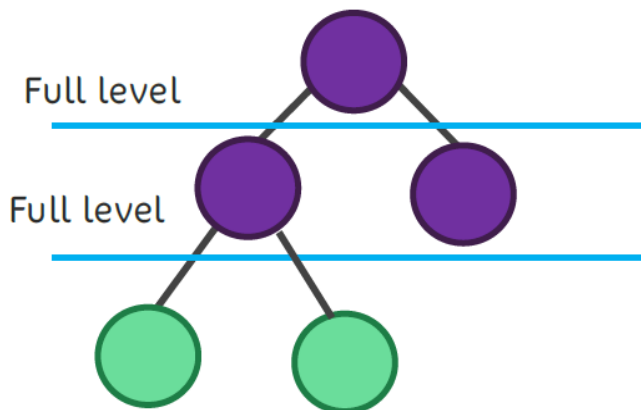
لدينا نوعان من ال Heap :

Max-Heap and Min-Heap

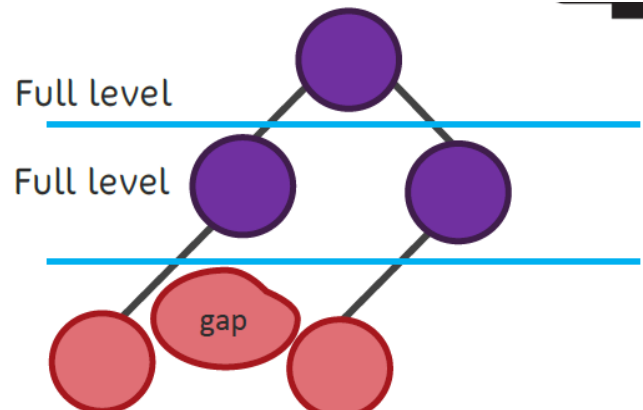
- في ال Max-Heap : لكل عقدة في الشجرة الشائبة يجب ان تكون قيمة كل أب أكبر من أو تساوي قيم أبنائه، وبالتالي يكون أكبر عنصر دائماً في الجذر.
- في ال Min-Heap : لكل عقدة في الشجرة الشائبة يجب ان تكون قيمة كل أب أصغر من أو تساوي قيم أبنائه، مما يجعل أصغر عنصر دائماً في الجذر.

ملاحظة هامة : العقد التي ليس لها أبناء تعتبر Heaps ، (الأوراق لا تخرق خاصية ال heap التي ذكرناها أعلاه) :

Complete Binary Tree



Valid level its filled starting from the left until a certain point

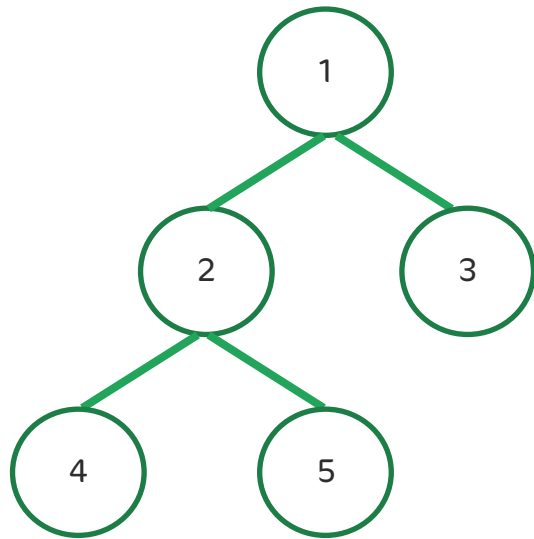


Invalid level its not filled starting from the left until a certain point (there is a gap)

في مقررنا عند استخدام مصطلح HEAP نحن نقصد تحديداً ال BINARY HEAP

■ التمثيل بالمصفوفة في ال Heap :

تم تمثيل ال Binary Heap باستخدام المصفوفات لأن ال heap عبارة عن شجرة ثنائية كاملة، وهذا يعني أن العقد ترتب دون وجود فراغات بينها، مما يجعل تخزينها داخل مصفوفة أمرًا فعالاً من ناحية الذاكرة. فبدلاً من استخدام مؤشرات لكل ابن كما في التمثيل التقليدي للأشجار، والتي تستهلك مساحة إضافية قد تصل إلى 8 بايت لكل مؤشر، فإن الاعتماد على المصفوفة يلغي الحاجة لهذه المؤشرات تماماً ويقلل من الهدر في الذاكرة. كما يمكن استخدام المصفوفات الديناميكية `vector` أو `ArrayList` ، والتي تقوم داخلياً بإنشاء مصفوفة جديدة أكبر حجماً عند امتلاء الحالية ونقل العناصر إليها، مما يسمح بالتوسع بسهولة دون تدخل المبرمج. وبفضل انتظام الهيكل، يمكن الوصول إلى الأبناء والآباء باستخدام علاقات بسيطة بين الفهارس: الابن الأيسر في $2i+1$ والابن الأيمن في $2i+2$ وذلك بافتراض أن الجذر موجود في الفهرس 0. هذا الأسلوب يجعل التمثيل بالمصفوفة الخيار الأمثل لبناء ال heap وتنفيذ عملياته بكفاءة عالية.



INDEX	0	1	2	3	4
VALUE	1	2	3	4	5



بناء Heap من مصفوفة معطاة :

عندما تكون لدينا مصفوفة أحادية تمثل شجرة ثنائية مكتملة (Complete Binary Tree) ، يمكن تحويلها إلى Heap من خلال التحقق من خصائصه. سنفترض هنا أننا نبني Max Heap ، بينما يعمل Min Heap بنفس الفكرة مع عكس المقارنات.

تعتمد عملية البناء على إجراء خطوة أساسية تُسمى Heapify ، هدفها هو إصلاح شجرة فرعية واحدة (subtree) بحيث يصبح جذر هذه الشجرة الفرعية ملتزماً بخاصية ال heap ، أي أكبر من أبنائه في ال Max-Heap أو أصغر منهم في ال Min-Heap .

كيف تُنفذ عملية Heapify في ال Max Heap ؟

نقارن العقدة الحالية مع أبنائها.

إذا كان أحد الأبناء أكبر من العقدة، نحدد الابن الأكبر ثم نقوم بتبديل القيم بينه وبين الأب.

بعد التبديل ننتقل إلى الشجرة الفرعية التابعة للابن الذي تم التبديل معه، ونعيد تنفيذ العملية عليه.

نستمر في التكرار حتى نصل إلى الأوراق أو إلى نقطة لا يحدث فيها أي تبديل.

عند حدوث تبديل، قد تختل خاصية ال Heap داخل الشجرة الفرعية المتأثرة، لذلك نكرر العملية لضمان أن جميع الأشجار الجزئية ما زالت تحقق خاصية ال Heap .

متى تكفي عملية Heapify واحدة ؟

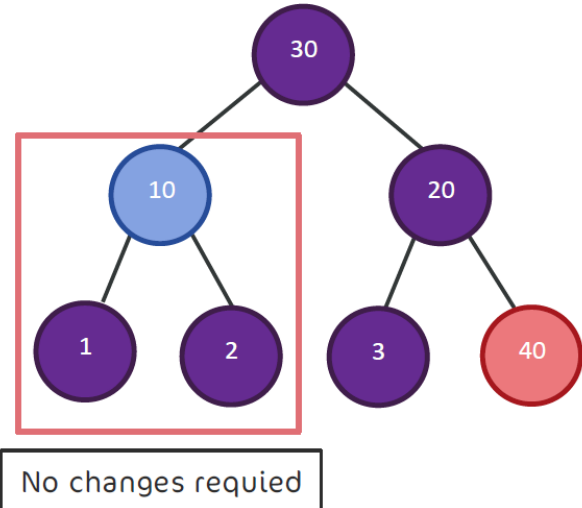
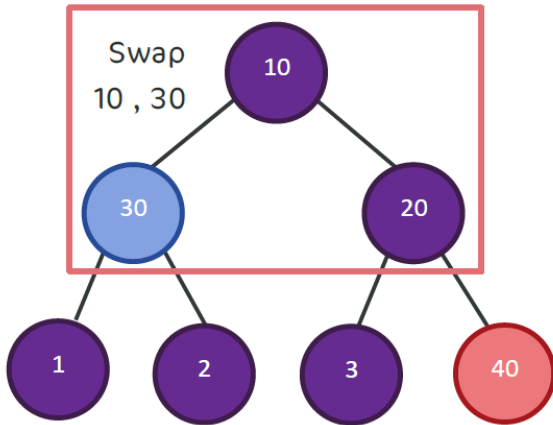
إذا كانت جميع الأشجار الفرعية تحقق خاصية ال Heap وكان الخلل موجوداً فقط عند الجذر، فإن تنفيذ عملية Heapify مرة واحدة على الجذر يكفي لإصلاح البنية وإعادة العنصر الأكبر إلى مكانه الصحيح.

ماذا لو لم تكن الشجرة Heap أساساً؟

في هذه الحالة نقوم ببناء ال Heap من الأسفل إلى الأعلى، أي من العقد القريبة من الأوراق وصولاً إلى الجذر.

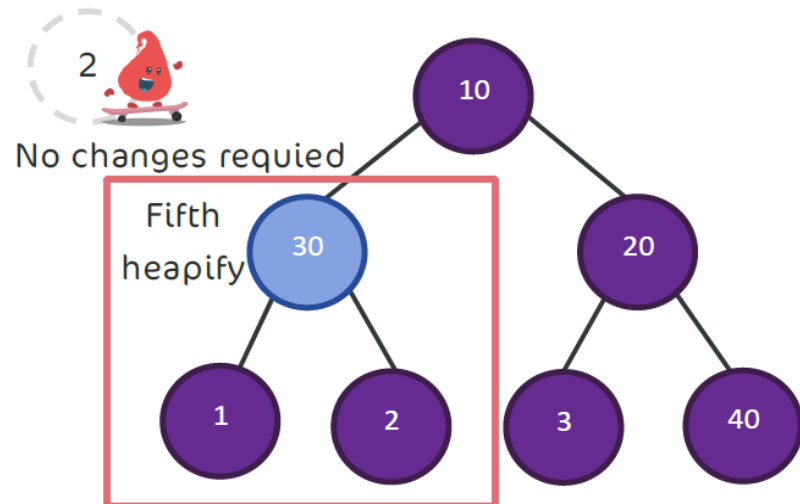
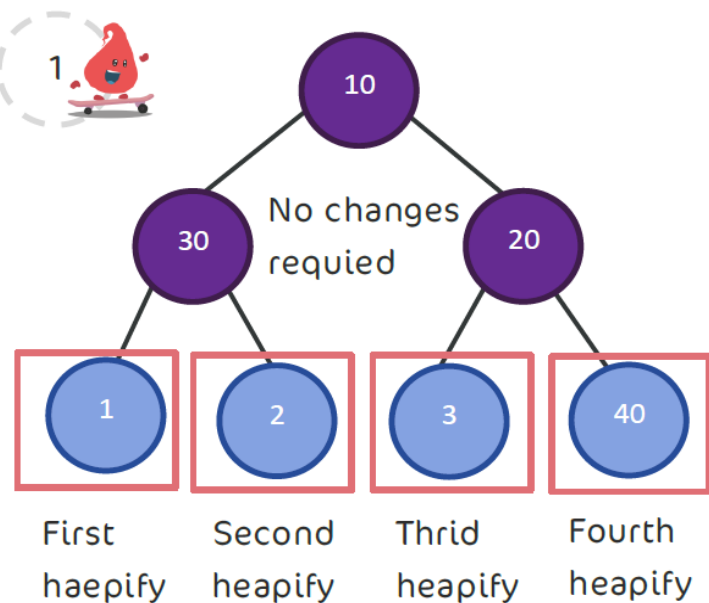
السبب في ذلك هو أن عملية Heapify تعتمد على مقارنة العقدة مع أبنائها، وبالتالي يجب أولاً التأكد من أن الأشجار الفرعية للأبناء تحقق خاصية ال Heap عندها فقط يمكن مقارنة الأب مع أبنائه بشكل صحيح.

وباتباع هذا الترتيب، تنتقل القيم الأكبر تدريجياً نحو الأعلى حتى تصل إلى الجذر، وبذلك تتحول الشجرة بالكامل إلى Max Heap صحيح.

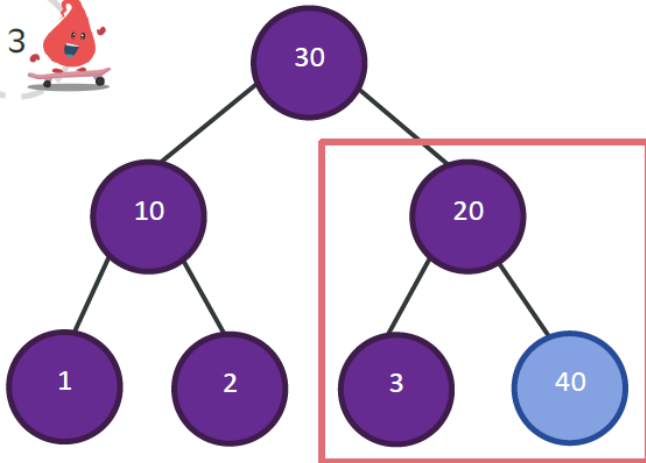


مثال مبسط على المشكلة :

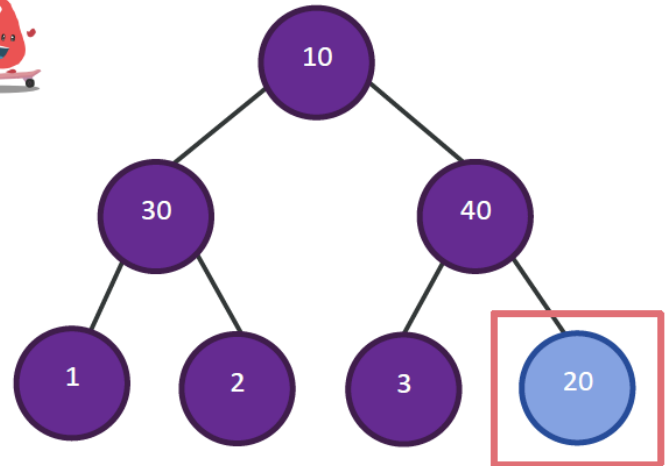
لو بدأنا بتطبيق العملية من الأعلى للأسفل Heapifying
الآن مهمما قمنا باستدعاء التابع على الأبناء وصولا للأوراق لن تصعد ال 40 الى الجذر وكناتج نهائي
بعد استدعاء التابع على جميع العقد الأبناء سيكون الناتج هو الشجرة التالية وهذا ليس heap
بينما سنعود لنفس الشجرة ونعيد تنفيذ التعليمات من الأسفل للأعلى



3

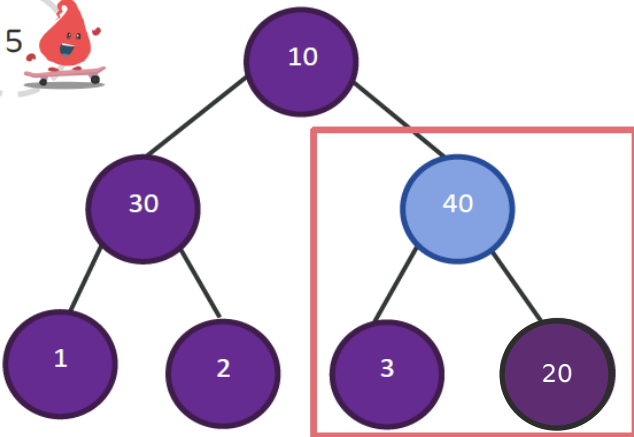


4

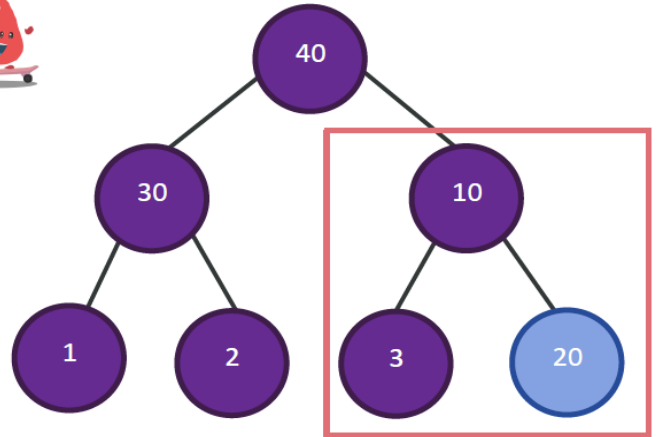


As we mentioned we call the function on the affected branch of the tree recursively (In this specific case no updates required so we stop here)

5

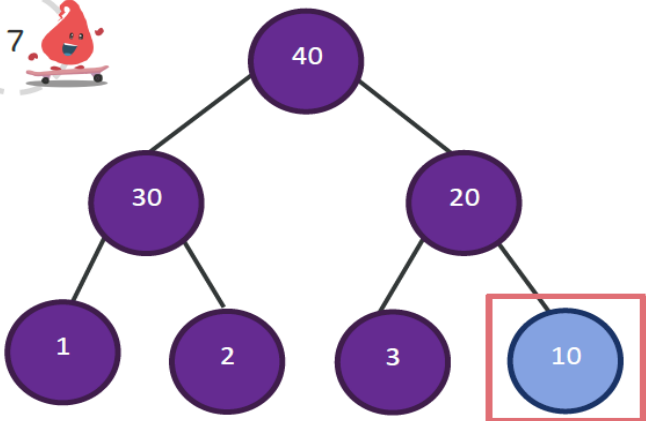


6



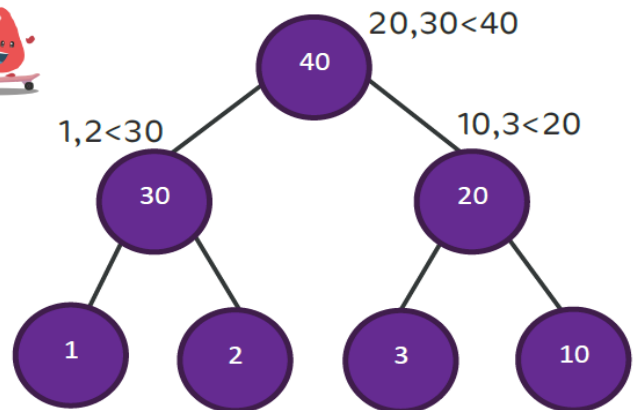
As we mentioned we call the function on the affected branch of the tree recursively (we need to swap 10 with 20 then call the function on the right child again).

7



No changes needed so we stop here

8



Leaf nodes are always valid heaps

Final result and it's a valid heap

ملاحظة هامة :

سنستدعي التابع كما قلنا من الأسفل إلى الأعلى، ولكن يمكننا استثناء العقد الورقية كون العقد الورقية تكون دائماً Valid Heaps ولن ننفذ أي عمليات عليها بحد ذاتها. لذلك يمكننا البدء باستدعاء التابع على أول عقدة غير ورقية (داخلية) ضمن الشجرة وصولاً إلى الجذر.

يمكننا تحديد فهرس (index) أول عقدة تمتلك أبناء — أي أول عقدة غير ورقية — باستخدام العلاقة:

$(n/2)-1$ حيث n هو عدد العقد الكلي في الشجرة

بعد فهم مبدأ العملية سنكتب code يقوم بما ذكرناه مسبقاً

```
import java.util.ArrayList;

public class Main {

    public static void heapify(ArrayList<Integer> arr,int i,int size) {
        // indexes setup
        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        // if the left child is larger
        if (left<size&&arr.get(left)>arr.get(largest)){
            largest = left;
        }
        // if the right child is larger
        if (right < size && arr.get(right) > arr.get(largest)) {
            largest = right;
        }
        // if the largest element is one of the children of the root
        if (largest != i) {
            //swap it with the root
            int temp = arr.get(i);
            arr.set(i, arr.get(largest));
            arr.set(largest, temp);
            //call the hepify on the affected branch
            heapify(arr, largest, size);
        }
    }
}
```

1



```

public static void buildMaxHeap(ArrayList<Integer> arr) {
    int size = arr.size();
    //starting from the first non leaf node until the root of the tree
    for (int i = size / 2 - 1; i >= 0; i--) {
        //heapify each node
        heapify(arr, i, size);
    }
}

public static void main(String[] args) {
    ArrayList<Integer> arr = new ArrayList<>();
    arr.add(3);
    arr.add(1);
    arr.add(6);
    arr.add(5);
    arr.add(2);
    arr.add(4);
    System.out.println("before building max-heap " + arr);
    buildMaxHeap(arr);
    System.out.println("after building max-heap " + arr);
}

```

before building max-heap

[3, 1, 6, 5, 2, 4]

after building max-heap

[6, 5, 4, 1, 2, 3]

بعد أن تحدثنا عن كيفية بناء الـ Heap من مصفوفة تمثل شجرة ثنائية مكتملة، يمكننا الآن تقدير تعقيد العملية السابقة. من النظرة الأولى يمكن التفكير بأن دالة heapify تعمل على عقدة معينة نزولاً حتى الوصول إلى الأوراق، وهذا في أسوأ حالة يحتاج إلى:

 $O(h)$ حيث h هو ارتفاع العقدة التي نقف عندها.

وبما أننا نستدعي التابع على حوالي:

$$\frac{n}{2}$$

عقدة تقريباً (ونُهمل عامل $1/2$ ونعتبر العدد n عقدة)، فقد يبدو أننا نقوم بما يقارب:

$$n \times h$$

عملية

وإذا افترضنا أن $h \approx \log(n)$

فقد نستنتج بشكل خاطئ أن التعقيد هو:

$$O(n \log n)$$

لكن هذا التحليل غير دقيق.

عند القيام بتحليل أدق، نلاحظ أن كلما ابتعدنا عن الجذر قل ارتفاع العقدة، وبالتالي يقل عدد العمليات المطلوبة لإصلاحها. كما أن عدد العقد الأبعد عن الجذر أكبر بكثير من عدد العقد القريبة منه، لأن عدد العقد يزداد بشكل أسّي كلما نزلنا في المستويات.

بالتالي معظم العقد تحتاج عدداً قليلاً جداً من العمليات لإصلاحها، أي تقريباً: $O(1)$

عدد قليل من العقد (القريبة من الجذر) يحتاج ما يقارب : $O(\log n)$

عند جمع العمليات المطلوبة على جميع الارتفاعات، نجد أن مجموع العمليات الكلي يتوزع بحيث تكون الكلفة الإجمالية لبناء الـ Heap خطية :

$$O(n)$$

العمليات التي يمكن تنفيذها على الـ Heap :

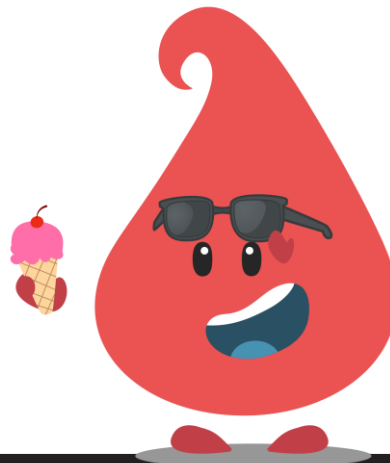
1. الـ Insert :

تعدّ عملية إدراج عنصر جديد في الـ Heap من العمليات الأساسية التي يجب تنفيذها بطريقة دقيقة حتى لا تتأثر خاصية Heap. فبما أن Heap يعتمد على ترتيب هرمي يحافظ على علاقة معينة بين الأب وأبنائه، فإن أي عنصر جديد يتم إدخاله قد يسبب خللاً في هذا الترتيب إذا لم تتم معالجته بالشكل الصحيح.

نبدأ عملية الإدراج بإضافة العنصر الجديد في آخر موقع في المصفوفة، أي في نهاية الشجرة المكتملة. هذا الموقع هو المكان الوحيد الذي يمكن أن نضيف فيه عنصراً جديداً دون كسر خاصية "الشجرة المكتملة" التي تعدّ شرطاً أساسياً في بنية الـ Heap .

بعد وضع العنصر في هذا الموقع، نلاحظ أن جميع العقد السابقة في الـ Heap ما زالت تحافظ على الخاصية باستثناء هذا العنصر الجديد فقط.

لذلك نحتاج إلى عملية خاصة تعيد الترتيب من الأسفل إلى الأعلى، وتعرف هذه العملية باسم HeapifyUp.



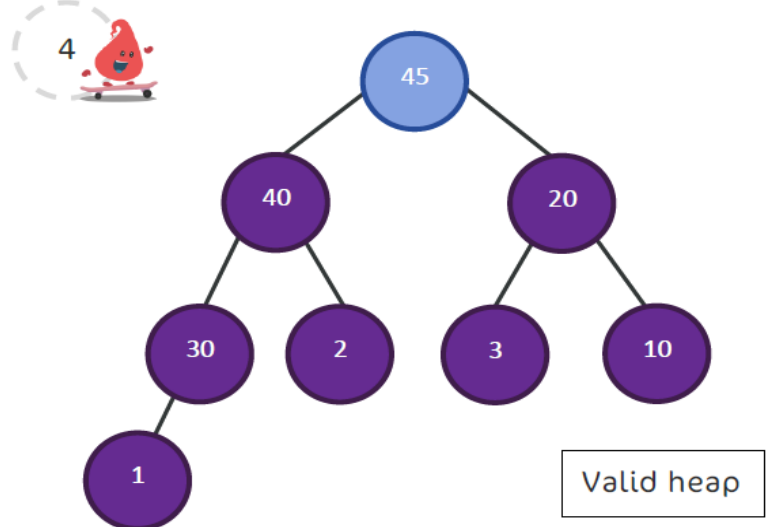
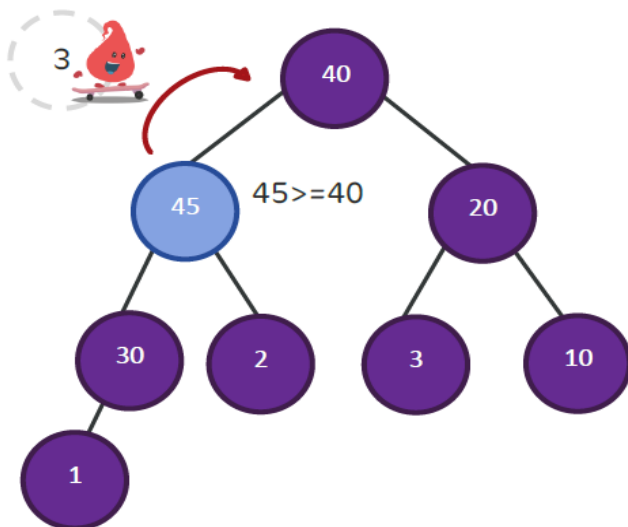
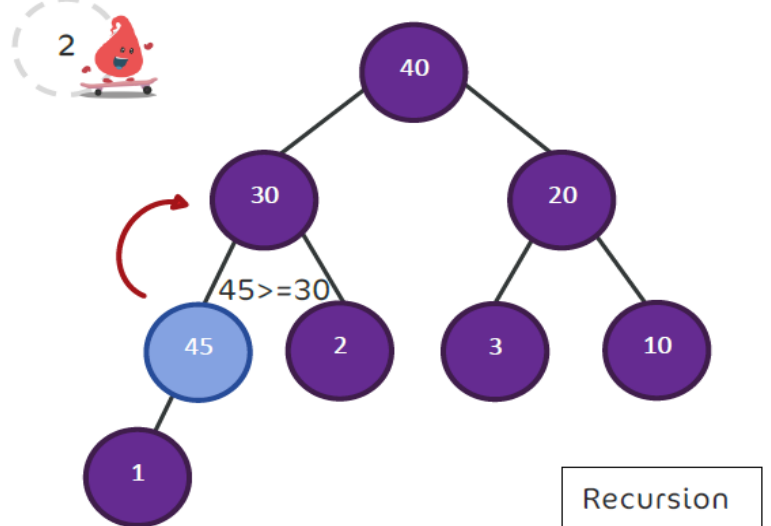
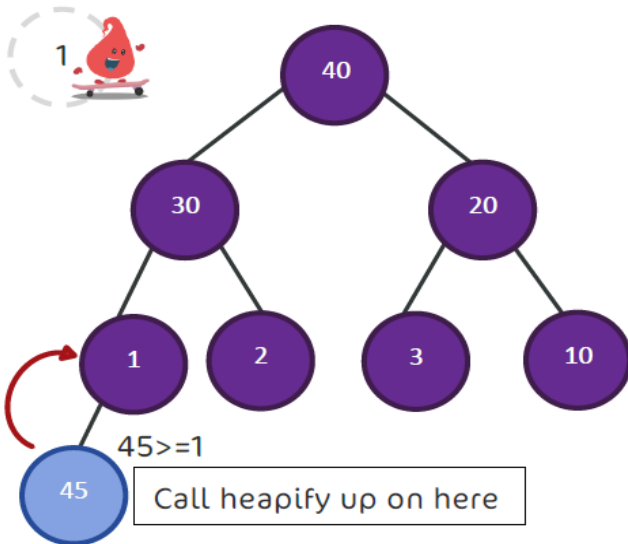
تعمل عملية HeapifyUp على مقارنة العنصر الجديد مع الأب المباشر له. فإذا كنا نتعامل مع Max-Heap، فإن الشرط الأساسي هو أن تكون قيمة الأب أكبر من أو تساوي قيمة الابن. فإذا وجدنا أن العنصر الجديد أكبر من أبيه، فهذا يعني أن موقعه الحالي غير صحيح، ويجب رفعه إلى مستوى أعلى. عندها نقوم بعملية تبديل بين العنصر الجديد وأبيه.

عند التبديل، لا نتوقف، بل نكرر العملية نفسها مع الأب الجديد، لأن التبديل قد يجعله في موقع لا يحقق شرط Heap مع أبيه الأعلى. وهكذا نستمر في الصعود خطوة بخطوة حتى نصل إلى أحد الحالتين:

إما أن يصل العنصر إلى الجذر، وفي هذه الحالة يكون قد أخذ مكانه الصحيح.

أو أن نصل إلى نقطة يصبح فيها الأب أكبر من الابن في (Max-Heap)، وبالتالي لا حاجة لمزيد من التبديل.

بهذه الطريقة، نضمن أن ال Heap يستعيد خاصيته بالكامل بعد إدراج أي عنصر جديد، دون التأثير على بقية العناصر التي كانت مرتبة مسبقاً.



عند تنفيذ عملية الحذف في بنية ال Heap ، نحتاج إلى التعامل معها بحذر شديد لأن إزالة أي عنصر وخاصة إذا كان العنصر المحذوف هو الجذر ، قد يؤدي إلى كسر خاصية ال Heap. لذلك تعتمد عملية الحذف على خطوات دقيقة تضمن الحفاظ على شكل الشجرة المكتملة من جهة، وعلى خاصية ال Heap من جهة أخرى.

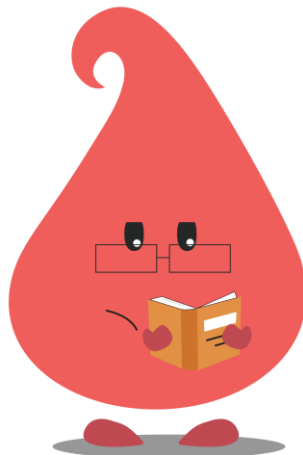
نبدأ عملية الحذف من خلال تبديل العنصر الذي نريد حذفه مع آخر عنصر في المصفوفة . هذا التبديل ضروري لأنه يسمح لنا بحذف العنصر الأخير بسهولة دون التأثير على شكل الشجرة. بعد التبديل، نقوم مباشرة بحذف آخر عنصر من المصفوفة، لأن الحذف من النهاية هو الأسهل والأكثر كفاءة.

لكن بعد هذه الخطوة، يصبح لدينا عنصر جديد وليكن X قد احتل موقع العنصر المحذوف. هذا العنصر X قد يكون في موقع غير مناسب، وقد يسبب خللاً في خاصية ال Heap. في هذه اللحظة، يكون الخلل محصوراً في نقطة واحدة فقط من الشجرة، وهي موقع العنصر X ، ولذلك نحتاج إلى تحديد ما إذا كان يجب رفع هذا العنصر للأعلى أو إنزاله للأسفل.

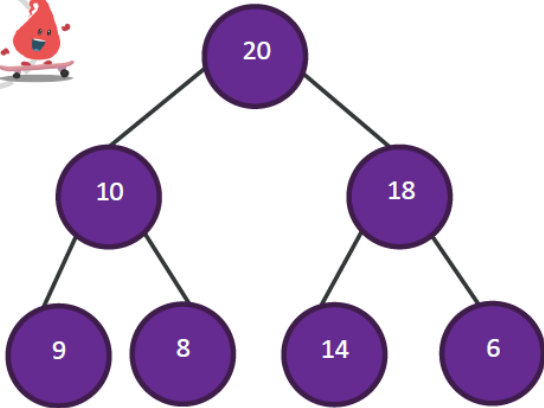
ولمعرفة الاتجاه الصحيح، نقوم أولاً بمحاولة تطبيق عملية heapifyUp على العنصر X. السبب في ذلك هو أن الشجرة فوق هذا الموقع ما زالت تحقق خاصية ال Heap ، وبالتالي إذا كان X أصغر من أبيه في (Min-Heap) أو أكبر من أبيه في (Max-Heap) ، فهذا يعني أنه يجب رفعه للأعلى. فإذا كان بالإمكان رفعه، نقوم بعملية التبديل مع الأب ونكرر العملية حتى يستقر في مكانه الصحيح.

أما إذا لم يكن بالإمكان رفع العنصر X للأعلى أي أن علاقته مع أبيه صحيحة فهذا يعني أن الخلل غير موجود في الجزء العلوي من الشجرة . وفي هذه الحالة، نتحقق مما إذا كان العنصر في موقع مناسب بالنسبة لأبنائه. فإذا لم يظهر أي خلل في علاقته مع الأبناء أيضاً، فهذا يعني أن العنصر X في موقعه الصحيح، وأن ال Heap قد عاد إلى وضعه الطبيعي دون الحاجة إلى أي تعديل إضافي.

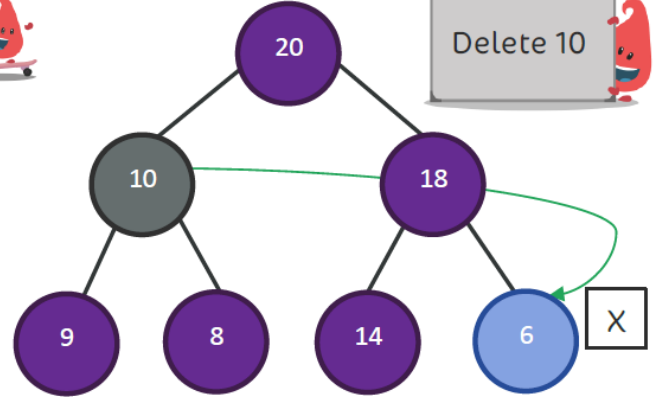
بهذه الخطوات الدقيقة، نضمن أن عملية الحذف تتم بكفاءة عالية، وأن ال Heap يستعيد توازنه بالكامل بعد إزالة أي عنصر.



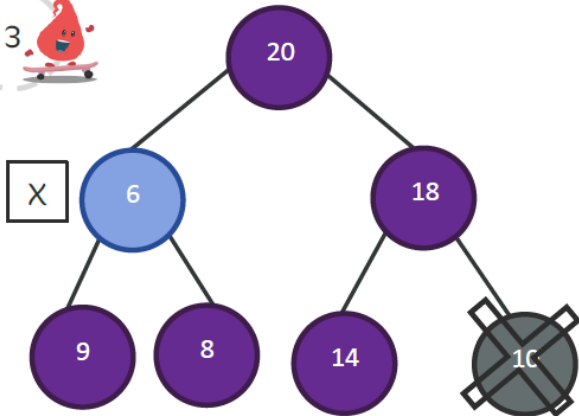
1



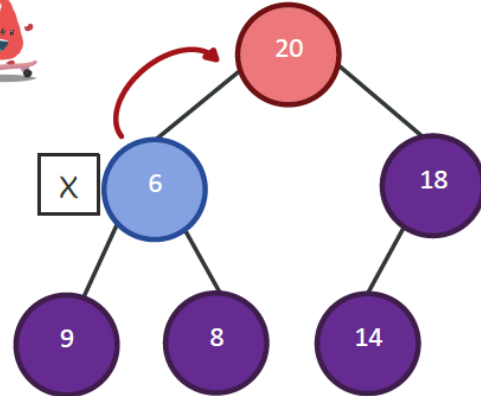
2



3

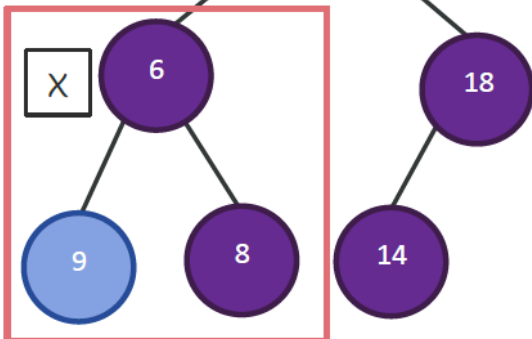


4



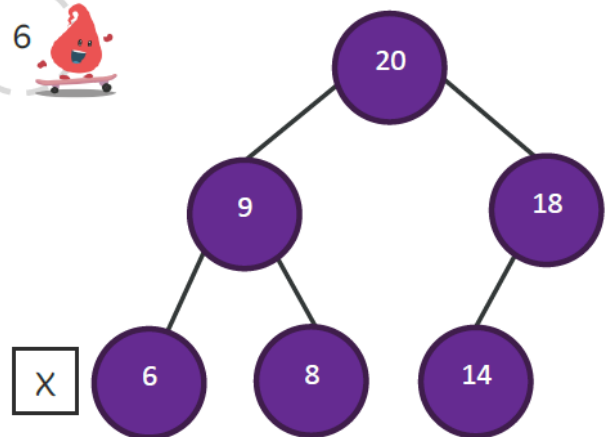
WE CAN'T HEAPIFY X UP

5

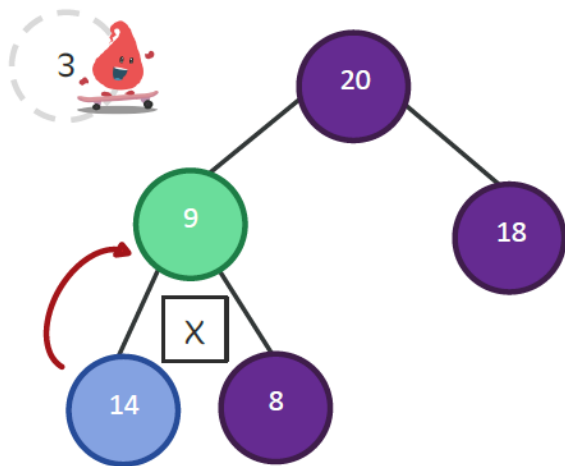
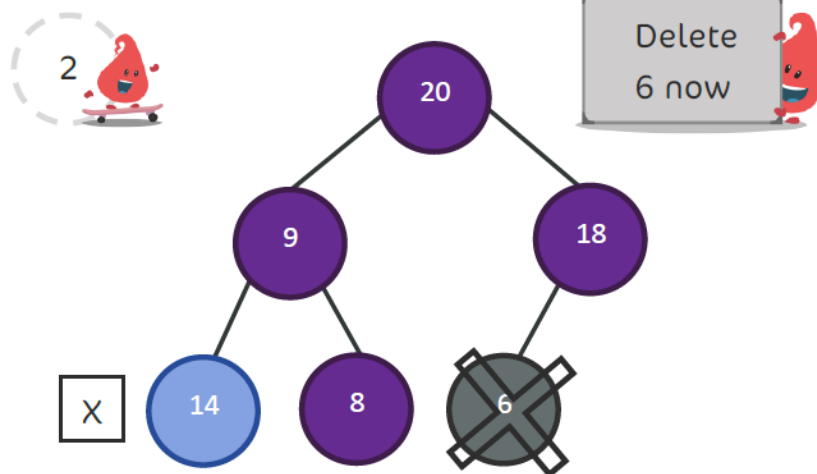
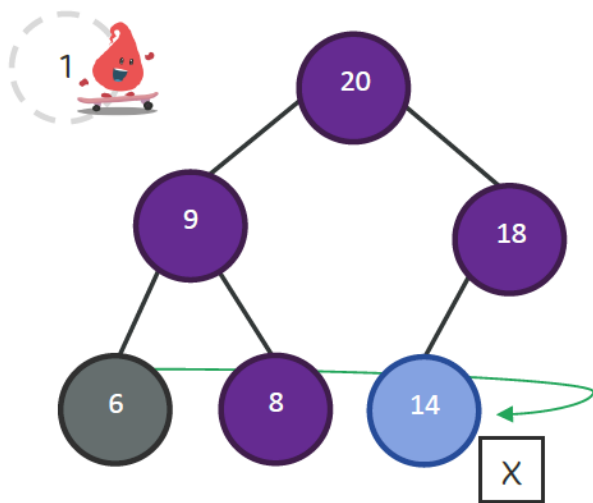


WE HEAPIFY X normally

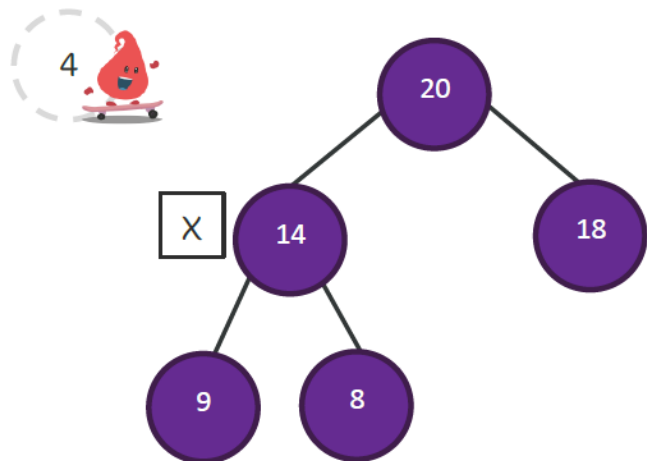
6



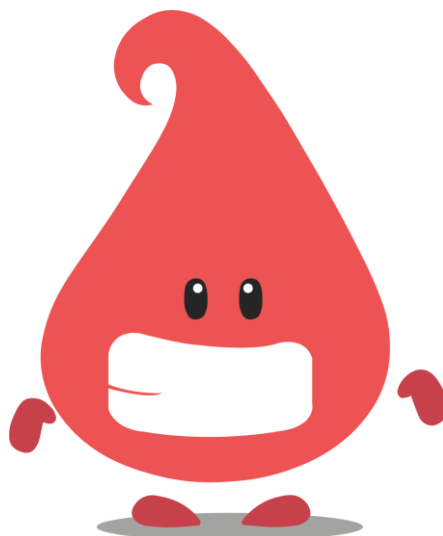
Now we are done and as we see it's a valid heap



WE CAN HEAPIFY X UP



Now everything is alright no need to use normal heapify



الآن سنستعرض الكود و كيف يمكن لنا القيام بالعمليتين السابقتين الحذف و الإضافة:

1

```
import java.util.ArrayList;
//class representing a max heap
class MaxHeap {
    // we will use an arraylist
    private ArrayList<Integer> heap=new
    ArrayList<>();
    //getting indexes of parent/left,right
    children (just to make it clear for you)
    private int parent(int i) {return (i - 1) / 2;}
    private int left(int i) {return 2 * i + 1;}
    private int right(int i) {return 2 * i + 2;}
    private void heapifyUp(int index) {
        // if we reached the root
        if (index == 0) return;
        //if the element is larger than it's parent then
        swap them and repeat the process on the
        parent
        if(heap.get(index)>heap.get(parent(index))){
            int temp = heap.get(index);
            heap.set (index,heap.get (parent (index)));
            heap.set(parent(index), temp);
            heapifyUp(parent(index));
        }
    }
    void heapify(int index) {
        // indexes and size setup
        int size = heap.size();
        int largest = index;
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        // if the left child is larger
        if(left<size&&heap.get(left)>heap.get(largest)){
            largest = left;
        }
    }
}
```

2

```
// if the right child is larger
if (right < size && heap.get(right) >
heap.get(largest)) {
    largest = right;
}
// if the largest element is one of the
children of the root
if (largest != index) {
    //swap it with the root
    int temp = heap.get(index);
    heap.set(index, heap.get(largest));
    heap.set(largest, temp);
    //call the heapify on the affected
    branch
    heapify(largest);
}
}
public void insert(int value) {
    //we just add the value at the last index
    heap.add(value);
    // then we heapify it up
    heapifyUp(heap.size() - 1);
}
```



```

public void delete(int value){
    //we search for the desired value first
    int index = heap.indexOf(value);
    //if we don't find it just return
    if (index == -1) return;
    //we swap between the last element in the heap and the one we want to delete
    int lastIndex = heap.size()-1;
    int temp = heap.get(index);
    heap.set(index, heap.get(lastIndex));
    heap.set(lastIndex, temp);
    //we remove the last element from the array
    heap.remove(lastIndex);
    //this condition guarantees that the value which we deleted is not the last element itself
    {so we don't get out of bounds exception}
    //(no need to repair the heap anyway in this case)

```

```

if (index < heap.size()) {
    //if the element is larger than its
    parent we heapify up
    if(index>0&&heap.get(index) >
    heap.get(parent(index))){
        heapifyUp(index);
    }
    // else we heapify normally
    else {
        heapify(index);
    }
}
}
public void printHeap() {
    System.out.println(heap);
}

```

```

public static void main(String[] args) {
    MaxHeap h = new MaxHeap();
    h.insert(20);
    h.insert(10);
    h.insert(18);
    h.insert(9);
    h.insert(8);
    h.insert(14);
    h.insert(6);
    System.out.print("normal heap:");
    h.printHeap();
    h.delete(6);
    System.out.print("after deleting 6:");
    h.printHeap();
    h.delete(10);
    System.out.print("after deleting 10:");
    h.printHeap();
}

```

هذا هو ال heap بصورة عامة، وتعرفنا على العمليات الأساسية به (يمكن تطبيق ذلك على ال min heap بمجرد التلاعب بالشروط).

خرج الكود:

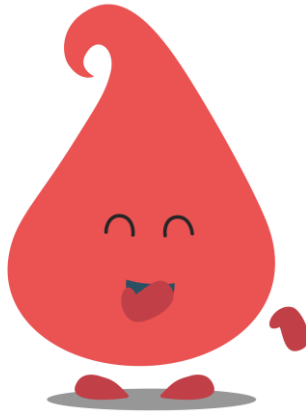
normal heap: [20, 10, 18, 9, 8, 14, 6]

after deleting 6: [20, 10, 18, 9, 8, 14]

after deleting 10: [20, 14, 18, 9, 8]

"خبير في تأجيل المهام" مهارة ليست مطلوبة في سوق

العمل..... فكمّل للأخير



اهم استخدامات ال Heap:

■ استخدام ال Heap في بناء ال Priority Queue :

يُعدّ heap من أهم الهياكل التي تُستخدم لبناء ال Priority Queue، وذلك لأنه يوفر طريقة فعّالة جدًا للتعامل مع العناصر التي تمتلك "أولوية" معينة. تعتمد فكرة ال Priority Queue على إمكانية استخراج أكبر عنصر أو أصغر عنصر بشكل دائم، وذلك حسب نوع الهيب المستخدم (Max-Heap) أو (Min-Heap).

الميزة الأساسية هنا هي أن الهيب يجعل عملية الوصول إلى العنصر الأهم عملية مباشرة وسريعة، لأن هذا العنصر يكون دائمًا موجودًا في الجذر.

كما أن عمليات الإضافة والحذف تتم بكفاءة عالية، حيث يمكن إدراج عنصر جديد بسهولة، ثم رفعه إلى موقعه الصحيح دون التأثير على بقية العناصر.

وبذلك يمكن القول إن ال Priority Queue ليست سوى هيب مُغلّف بعمليات مخصّصة، حيث يتم التركيز على الجذر فقط عند الحذف، مع استخدام عمليات الإصلاح للحفاظ على خاصية الهيب بعد كل تعديل.

■ تشابه الهيب مع خوارزمية Selection Sort :

تقوم خوارزمية Selection Sort التقليدية على مبدأ بسيط:

نبحث عن أصغر عنصر في المصفوفة ونضعه في بدايتها، ثم نستثني هذا العنصر ونبحث عن ثاني أصغر عنصر، وهكذا حتى يتم ترتيب جميع العناصر.

لكن المشكلة الأساسية في هذه الخوارزمية هي أنها تعتمد على البحث الخطي عن أصغر عنصر في كل خطوة، مما يجعلها بطيئة جداً عند التعامل مع عدد كبير من العناصر.

ففي كل موقع من مواقع المصفوفة، نحتاج إلى المرور على جميع العناصر المتبقية للعثور على الأصغر، وهذا يؤدي إلى تعقيد زمني من رتبة:

$$O(n^2)$$

وهو تعقيد غير مناسب للتطبيقات التي تتطلب أداءً عالياً.

■ ما علاقة خوارزمية Selection sort بموضوعنا اليوم؟

نلاحظ أننا نعتمد بصورة عامة على إيجاد العنصر الأصغر بشكل دوري ونستثني العناصر التي استخدمناها مسبقاً في هذه الخوارزمية، ونلاحظ أننا من خلال الـ min heap يمكننا تقديم أصغر عنصر، وعند الانتهاء منه يمكننا حذفه وسيعيد الـ min heap إصلاح نفسه وتقديم العنصر الأصغر الذي يليه، وهذا بتعقيد $O(\log(n))$ (تعقيد الـ heapify بالنسبة للجذر)

وبالتالي يمكننا تحسين خوارزمية الترتيب السابقة من خلال استخدام الـ heap ، فيصبح من أجل كل مكان في المصفوفة المكونة من n عنصر نحتاج لـ $O(\log(n))$ عملية حتى نصل للعنصر الأصغر، وبالتالي يصبح التعقيد الكلي:

$$O(n \log(n))$$

نوع الترتيب الجديد هذا يسمى heap sort